

GEP Intern – Technology: OA & Interview Cheat Sheet

Print this — desk reference for OA + R1 + R2

Ojas Srivastava | Mon 17 Aug 3PM PPT + OA | R1 Tue 18 | R2 Wed 19 | Hyd / Mumbai

1) Drive schedule

- **Mon 17:** PPT 3PM → proctored OA same day (MCQ + coding)
- **Tue 18:** R1 technical (DSA, OOP, resume)
- **Wed 19:** R2 functional / projects (+ HR)
- Before OA: print resume, webcam, stable net, warm-up 1 Easy + Kadane dry-run

2) OA — section map

AI MCQs (MAJOR)	~12–20 Q. Intro only: AI vs ML vs DL, supervised / unsupervised, linear regression idea, where CV / RL / NLP is used. Coursera-level. Do not overstudy.
OS + DBMS	Process vs thread, deadlock (4 conditions), paging, mutex vs semaphore. Keys, JOINS, GROUP BY, 1NF–3NF, ACID.
Coding	1 LC Easy–Medium: strings, hashing, Kadane, Two Sum. Comment approach; check edges.
JavaScript	Code given — fill in the blanks. == vs ===, map / filter, loops, strings.
Also possible	Aptitude (SI/CI, profit-loss, DI) + soft-skills / SJT. Time-box if present.

3) AI MCQs (intro only — no math)

- **AI** = broad field. **ML** = learn from data. **DL** = neural nets.
- **Supervised:** labeled data (classification / regression). **Unsupervised:** clusters / patterns (no labels).
- **Linear regression:** predicts a continuous value (price, delay) — it does not classify.
- **CV:** images / video (face, OCR). **RL:** agent + reward (games, robotics). **NLP:** text / speech (chatbots, translation).
- Bias = skewed training data. First instinct is enough — do not spiral into ML theory.

4) OS flash

- Process = program in execution. Thread = lightweight unit inside a process.
- **Deadlock (all 4):** mutual exclusion, hold & wait, no preemption, circular wait.
- **Paging:** memory split into fixed pages / frames — reduces external fragmentation.
- **Mutex:** one thread at a time. **Semaphore:** count-based access.
- Context switch = save / restore CPU state. Scheduling: preemptive vs non-preemptive.

5) DBMS flash

- **PK** uniquely identifies a row. **FK** references a PK. Candidate key = minimal unique set.
- **1NF** atomic columns. **2NF** no partial dependency on PK. **3NF** no transitive dependency.
- **ACID:** Atomicity, Consistency, Isolation, Durability.
- **Index:** faster reads, slower writes. B-tree idea for ordered lookup.
- **INNER JOIN** = matching rows both sides. **LEFT JOIN** = all left rows + match or NULL.
- **GROUP BY + HAVING:** aggregate, then filter groups. WHERE cannot filter on aggregates.

6) OOP (<2 min spoken)

- **Encapsulation** hide data. **Abstraction** what, not how. **Inheritance** reuse. **Polymorphism** same interface, different behavior.
- Access: public / private / protected.
- Stack = LIFO. Queue = FIFO. Linked list: O(1) insert at head, O(n) middle.
- BST: left < root < right. Graph: BFS = shortest unweighted path. DFS = explore / islands.

7) Named algorithms — 30s each

- **Kadane:** max subarray. Add to running sum; if sum < 0, reset to 0; track max.
- **Binary search:** sorted array; lo / hi / mid; O(log n).
- **Two Sum:** hash map of complement; O(n).
- **BFS:** queue; level-order; shortest path on unweighted graph.
- **DFS:** stack / recursion; islands, connectivity.
- **Sliding window:** fixed or variable window on array / string.
- **Prefix sum:** precompute for range queries. **Dijkstra:** weighted shortest path (greedy + priority queue).
- **Sorting:** Merge O(n log n) stable extra space. Quick avg O(n log n) pivot matters. Heap O(n log n) in-place. Selection O(n²) min swaps.

8) Kadane dry-run (memorize this)

- Array: [-2, 1, -3, 4, -1, 2, 1, -5, 4] → max = 6, subarray [4, -1, 2, 1]
- Running sum: -2, 1, -2, 4, 3, 5, 6, 1, 5. Max tracks 1, 4, 5, 6.

9) Sample OA patterns + JS + SQL

- **Alt case:** abcde + 10101 → AbCdE (1 = upper, 0 = lower)
- First unique char: freq map, one pass. Valid parens: stack. Reverse: two pointers.
- Primes 1–100: sieve or trial division. Fib in a range: iterate; watch overflow.
- **JS:** == coerces; === strict. map new array; filter subset; reduce aggregate. Read the full snippet before filling blanks.
- **SQL:** INNER JOIN authors a, books b ON a.id = b.author_id. LEFT JOIN keeps authors with 0 books.
- SELECT dept, COUNT(*) FROM emp GROUP BY dept HAVING COUNT(*) > 5

10) Aptitude (only if the section appears)

- SI = $PRT/100$. CI: amount = $P(1 + r/100)^n$; CI = amount – principal.
- Profit % = $(SP - CP)/CP \times 100$. Speed = distance / time. Opposite direction: add speeds.
- Blood relation: draw a family tree. Venn: sketch sets, do not guess.

11) Coding interview loop — never silent

- Clarify input / output + constraints → brute force (say complexity) → optimize (hash, two pointers, BS, Kadane)
- Code cleanly → dry-run 2 examples + edges (empty, one element, duplicates) → time / space
- If stuck: repeat the problem, start brute force, ask for a hint on approach. Partial credit beats silence.

12) R1 likely questions

- OOP pillars. Stack vs queue vs list vs tree. Middle of linked list: slow / fast pointers.
- Merge vs Quick — when to use which. Python vs Java (typing, JVM, speed, use-case).
- Write: reverse array, primes to 100, Fibonacci in a range. Explain Kadane / Binary Search by name + dry-run.
- Binary search: `lo=0, hi=n-1; while lo <= hi: mid=(lo+hi)//2 ...`
- Level-order BFS: queue the root; pop; visit; push left then right.
- STL: vector, string, map / unordered_map, set, queue, stack. Python: dict, deque for BFS.

13) Projects — role + follow-ups

- **IFFCO**: 10+ REST APIs, 50+ daily workflows, Node / Express / MySQL, Docker / CI/CD. Follow-ups: validation? indexing? how do you debug prod?
- **LogiFlow**: co-lead; 15k records; Redis 100–400ms; pytest 100/100; GCP. Follow-ups: why Redis? what breaks without cache?
- **Community Hero**: solo; Gemini categorization + guardrails; Firestore; Top 20. Follow-ups: bad LLM output? why Firestore?
- **AirHelp**: RAG + ChromaDB; WebSockets; 50–100 users, 1–3s latency. Follow-ups: retrieval vs generation? bottlenecks?
- **Cursor (if asked)**: daily Agent / Composer. Resume Optimiser: 966 companies, 10.7k+ contacts, OA Forge 14k+ questions / 5.5k+ tests, 8 CI workflows, 28k+ Python LOC, 69 resumes + 26 cover letters, 446 commits.

14) GEP pitch + HR

- GEP = AI-powered **procurement & supply chain** (software + consulting + managed services).
- Products: **Quantum Intelligence**, **SMART**, **NEXXE** (Azure SaaS). Open to Angular / .NET / Azure.
- Why SWE from AI: I ship production systems; AI is how I add intelligence on top.
- **Location**: pick Hyderabad or Mumbai and stick. Strength: ownership design → deploy. Weakness: one real one + how you manage it.
- End Q: “How do interns contribute to AI agent workflows on procurement data?”
- Layman pitch: “I build tools that automate daily work and turn messy data into decisions.”

15) Proctoring + Mon morning 15 min

- Charged laptop, webcam / mic, quiet room, ID, stable net. No tab switch, no phone, no discussing OA.
- No new research tabs day-of. No hard CP grind. No resume lies. Sleep well Monday night — R1 is Tuesday.
- Warm-up: Two Sum or first unique char (10 min). Kadane dry-run aloud (3 min). OOP pillars + one JOIN (2 min). Location locked.
- Defend every buzzword you say: Node.js, Express, FastAPI, Redis, pytest, GCP, Firestore, Gemini, RAG, ChromaDB, Docker, CI/CD, WebSockets, JOIN, ACID.

Golden rule: Clarify → approach → code → dry-run → complexity. Never silent.